

# Lecture 07: Logistic Regression

## Introduction to Machine Learning

Sajjad Amini

Department of Electrical Engineering  
Sharif University of Technology

# Contents

- 1 Approach Definitions
- 2 Binary Logistic Regression
- 3 Multinomial Logistic Regression
- 4 Bayesian Logistic Regression

Except explicitly cited, the reference for the material in slides is:

- Murphy, K. P. (2022). *Probabilistic machine learning: an introduction*. MIT press.

# Section 1

## Approach Definitions

## Logistic Regression

Logistic regression is discriminative classification model  $p(y|\mathbf{x};\boldsymbol{\theta})$  (supervised learning) where:

|                               |                              |
|-------------------------------|------------------------------|
| $\mathbf{x} \in \mathbb{R}^D$ | Fixed dimension input vector |
| $y \in \{1, \dots, C\}$       | Class label                  |
| $\boldsymbol{\theta}$         | Model parameters             |

Based on the value of  $C$ , we have:

|         |                                      |
|---------|--------------------------------------|
| $C = 2$ | Binary logistic regression (BLR)     |
| $C > 2$ | Multiclass logistic regression (MLR) |

## Section 2

# Binary Logistic Regression

# Binary Logistic Regression

## Model

Model:

$$p(y|\mathbf{x}; \boldsymbol{\theta}) = \text{Ber}(y|\sigma(\mathbf{w}^T \mathbf{x} + b))$$

where:

|                                         |                  |
|-----------------------------------------|------------------|
| $\sigma(\cdot)$                         | Sigmoid function |
| $\mathbf{w}$                            | Weight vector    |
| $b$                                     | Bias value       |
| $\boldsymbol{\theta} = [b; \mathbf{w}]$ | Model parameters |

## Label Set

Define *logit*  $a = \mathbf{w}^T \mathbf{x} + b$ .

- If  $y \in \{0, 1\}$  then 
$$\begin{cases} p(y = 1|\mathbf{x}; \boldsymbol{\theta}) = \sigma(a) \\ p(y = 0|\mathbf{x}; \boldsymbol{\theta}) = 1 - \sigma(a) = \sigma(-a) \end{cases}$$
- If  $\tilde{y} \in \{-1, 1\}$  then  $p(\tilde{y}|\mathbf{x}; \boldsymbol{\theta}) = \sigma(\tilde{y}a)$

# Decision Boundary

## Decision Boundary for Binary Classification

Decision boundary corresponds to the point  $\mathbf{x}^* \in \mathbb{R}^D$  where  $p(y = 1|\mathbf{x} = \mathbf{x}^*; \boldsymbol{\theta}) = 0.5$ .

## Decision Boundary

We want to find function  $g(\mathbf{x})$  that outputs 1 if  $y = 1$  is more probable and 0 otherwise. Thus:

$$g(\mathbf{x}) = \mathbb{I}(p(y = 1|\mathbf{x}; \boldsymbol{\theta}) > p(y = 0|\mathbf{x}; \boldsymbol{\theta})) = \mathbb{I}\left(\log \frac{p(y = 1|\mathbf{x}; \boldsymbol{\theta})}{p(y = 0|\mathbf{x}; \boldsymbol{\theta})} > 0\right) = \mathbb{I}(a > 0)$$

Thus decision boundary is:

$$f(\mathbf{x}; \boldsymbol{\theta}) = b + \langle \mathbf{w}, \mathbf{x} \rangle = 0$$



# Decision Boundary

## Decision Boundary Characterization

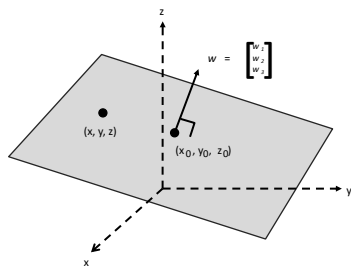
We know point on the hyperplane must satisfy  $\mathbf{w}^T(\mathbf{x} - \mathbf{x}_0) = 0$  where  $\mathbf{x}_0$  is a vector on the hyper plane and  $\mathbf{w}$  is normal vector. Thus:

Decision boundary is a hyperplane with normal vector  $\mathbf{w}$  and  $b = -\langle \mathbf{w}, \mathbf{x}_0 \rangle$

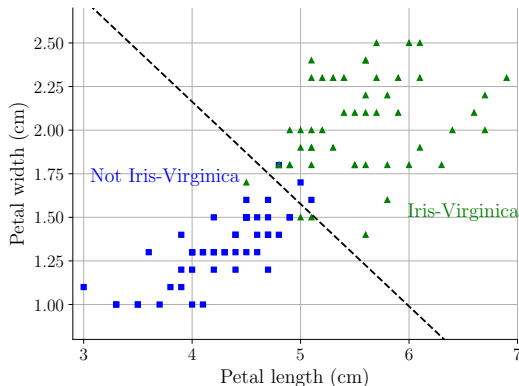
## Linearly Separable

If we can perfectly separate the training samples of a binary classification problem using a hyperplane, then the problem is known as linearly separable.

# Decision Boundary



(a) Decision boundary in 3D space



(b) Decision boundary for Iris-Virginica flower

# Feature Transformation

## Nonlinear Decision Boundary

Assume  $\phi(\cdot) : \mathbb{R}^D \rightarrow \mathbb{R}^{D'}$  represents a feature transformer. As an example consider:  $\phi(x_1, x_2) = [1, x_1^2, x_2^2]$ . Let  $\mathbf{w} = [-R^2, 1, 1]$ . Then decision boundary is:

$$\langle \mathbf{w}, \phi(\mathbf{x}) \rangle = 0$$

which represents a circle (nonlinear decision boundary).

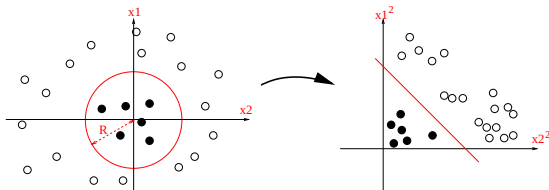


Figure: Nonlinear decision boundary for BLR

## Reformulating logit

$$a = \langle \mathbf{w}, \mathbf{x} \rangle + b = \langle [b, \mathbf{w}], [1, \mathbf{x}] \rangle, \begin{cases} [b, \mathbf{w}] : \text{Augmented weight vector} \\ [1, \mathbf{x}] : \text{Augmented input feature} \end{cases}$$

## NLL

Assume  $\mu_n = \sigma(a_n)$  and  $y \in \{0, 1\}$ , then:

$$\begin{aligned} \text{NLL}(\mathbf{w}) &= -\frac{1}{N} \log p(\mathcal{D}|\mathbf{w}) = -\frac{1}{N} \log \prod_{n=1}^N \text{Ber}(y_n|\mu_n) \\ &= -\frac{1}{N} \sum_{n=1}^N [y_n \log \mu_n + (1 - y_n) \log(1 - \mu_n)] = \frac{1}{N} \sum_{n=1}^N \mathbb{H}(y_n, \mu_n) \end{aligned}$$

## Gradient vector

$$\mathbf{g}(\mathbf{w}) = \nabla_{\mathbf{w}} \text{NLL}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N (\mu_n - y_n) \mathbf{x}_n = \frac{1}{N} (\mathbf{1}_N^T (\text{diag}(\boldsymbol{\mu} - \mathbf{y}) \mathbf{X}))^T$$

$$\text{where } \mathbf{X} = \begin{bmatrix} - & \mathbf{x}_1^T & - \\ \vdots & & \\ - & \mathbf{x}_N^T & - \end{bmatrix}, \quad \boldsymbol{\mu} = \begin{bmatrix} \mu_1 \\ \vdots \\ \mu_N \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_N \end{bmatrix}.$$

## Hessian Matrix

$$\mathbf{H}(\mathbf{w}) = \nabla_{\mathbf{w}} \nabla_{\mathbf{w}}^T \text{NLL}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N (\mu_n (1 - \mu_n) \mathbf{x}_n) \mathbf{x}_n^T = \frac{1}{N} \mathbf{X}^T \mathbf{S} \mathbf{X}$$

$$\text{where } \mathbf{S} = \text{diag}(\mu_1(1 - \mu_1), \dots, \mu_N(1 - \mu_N)).$$

# Characterization of $\text{NLL}(\mathbf{w})$

$\mathbf{H}(\mathbf{w})$  is PD

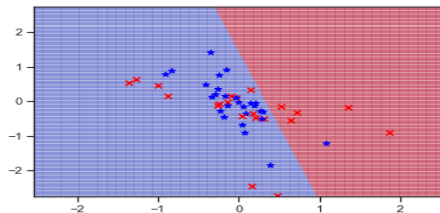
$$\forall \mathbf{v} : \mathbf{v}^T \mathbf{H}(\mathbf{w}) \mathbf{v} = \frac{1}{N} \mathbf{v}^T \mathbf{X}^T \mathbf{S} \mathbf{X} \mathbf{v} = \frac{1}{N} (\mathbf{S}^{\frac{1}{2}} \mathbf{X} \mathbf{v})^T (\mathbf{S}^{\frac{1}{2}} \mathbf{X} \mathbf{v}) = \frac{1}{N} \|\mathbf{S}^{\frac{1}{2}} \mathbf{X} \mathbf{v}\|_2^2 > 0$$

provided  $N(\mathbf{S}^{\frac{1}{2}} \mathbf{X}) = \{\mathbf{0}\}$

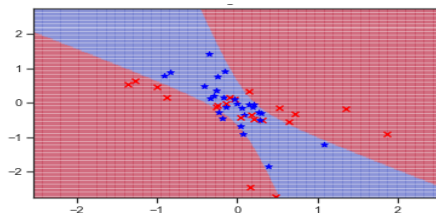
Global minimizer

Thus  $\text{NLL}(\mathbf{w})$  is twice differentiable and its hessian matrix is PSD. Thus  $\text{NLL}(\mathbf{w})$  is convex and stationary point  $\mathbf{w}^*$  ( $\mathbf{g}(\mathbf{w}^*)$ ) is the global minimizer.

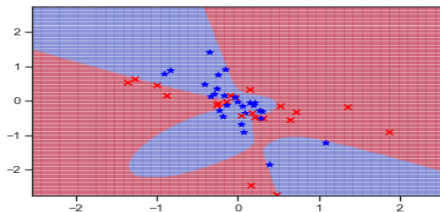
# Overfitting Problem



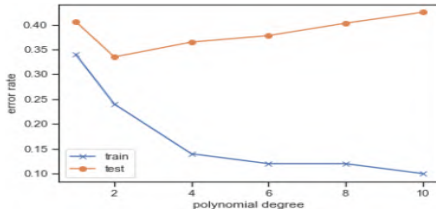
(a)  $K = 1$



(b)  $K = 2$



(c)  $K = 4$



(d) Error vs Complexity

Figure: Overfitting of BLR model when increasing the transformation complexity

## Weights Amplitude vs Model Complexity

$$K = 1 \Rightarrow \hat{\mathbf{w}}_{mle} = (0.513, 0.119)$$

$$K = 2 \Rightarrow \hat{\mathbf{w}}_{mle} = (2.275, 0.060, 11.842, 15.403, 2.512)$$

$$K = 4 \Rightarrow \hat{\mathbf{w}}_{mle} = (-3.078, \dots, -9.032, 51.771, 10.250)$$

Overfitting is accompanied by increasing the amplitude of weights.

*Solution:* One solution is to add a zero-mean Gaussian prior as  $p(\mathbf{w}) = \mathcal{N}(\mathbf{w}|\mathbf{0}, C\mathbf{I})$



## Objective Function

Using MAP estimation we have the following objective function:

$$\text{PNLL}(\mathbf{w}) = \text{NLL}(\mathbf{w}) + \lambda \|\mathbf{w}\|_2^2$$

- The above formulation is called  $\ell_2$  regularization or *Weight Decay*.

## Hyper-parameter Effect

Based on lambda:

- $\lambda \uparrow \Rightarrow$  more penalization  $\Rightarrow$  less flexible model
- $\lambda \downarrow \Rightarrow$  less penalization  $\Rightarrow$  more flexible model

## Derivatives

In this case, the derivatives are calculated as:

$$\begin{aligned}\text{PNLL}(\mathbf{w}) &= \text{NLL}(\mathbf{w}) + \lambda \mathbf{w}^T \mathbf{w} \\ \nabla_{\mathbf{w}} \text{PNLL}(\mathbf{w}) &= \mathbf{g}(\mathbf{w}) + 2\lambda \mathbf{w} \\ \nabla_{\mathbf{w}}^2 \text{PNLL}(\mathbf{w}) &= \mathbf{H}(\mathbf{w}) + 2\lambda \mathbf{I}\end{aligned}$$

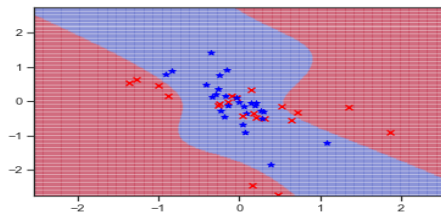
## Positive Definiteness of Hessian Matrix

Assume  $\lambda > 0$ , then:

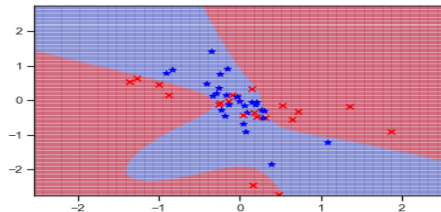
$$\begin{aligned}\forall \mathbf{v} : \mathbf{v}^T \nabla_{\mathbf{w}}^2 \text{PNLL}(\mathbf{w}) \mathbf{v} &= \mathbf{v}^T \mathbf{H}(\mathbf{w}) \mathbf{v} + 2\lambda \mathbf{v}^T \mathbf{I} \mathbf{v} = \frac{1}{N} \mathbf{v}^T \mathbf{X}^T \mathbf{S} \mathbf{X} \mathbf{v} + 2\lambda \|\mathbf{v}\|_2^2 \\ &= \frac{1}{N} \|\mathbf{S}^{\frac{1}{2}} \mathbf{X} \mathbf{v}\|_2^2 + 2\lambda \|\mathbf{v}\|_2^2 > 0\end{aligned}$$

$\nabla_{\mathbf{w}}^2 \text{PNLL}(\mathbf{w})$  is always PD.

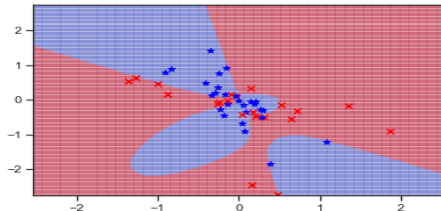
# Weight Decay Result



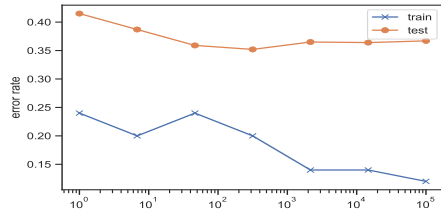
(a)  $C = 1$



(b)  $C = 316$



(c)  $C = 100000$



(d) Error vs inverse regularization

Figure: The effect of weight decay in BLR model performance

## Reason for Standardization

For MAP estimation, we use  $\mathcal{N}(\mathbf{w}|\mathbf{0}, \lambda^{-1}\mathbf{I})$  prior for weights. This prior implicitly assumes the input features to be similar in magnitude. To assure this, we can use the following methods:

- Individual normalization:

$$\hat{x}_{nd} = \frac{x_{nd} - \hat{\mu}_d}{\hat{\sigma}_d}, \begin{cases} \hat{\mu}_d = \frac{1}{N} \sum_{n=1}^N x_{nd} \\ \hat{\sigma}_d^2 = \frac{1}{N} \sum_{n=1}^N (x_{nd} - \hat{\mu}_d)^2 \end{cases}, d = 1, \dots, D$$

- Min-max scaling:

$$\hat{x}_{nd} = \frac{x_{nd} - m_d}{M_d - m_d}, \begin{cases} m_d = \min_n x_{nd} \\ M_d = \max_n x_{nd} \end{cases}, d = 1, \dots, D$$

- Data whitening using eigenvectors

## Section 3

# Multinomial Logistic Regression

# Multinomial Logistic Regression

## Model

Model:

$$p(y|\mathbf{x}; \boldsymbol{\theta}) = \text{Cat}(y|\mathcal{S}(\mathbf{W}\mathbf{x} + \mathbf{b}))$$

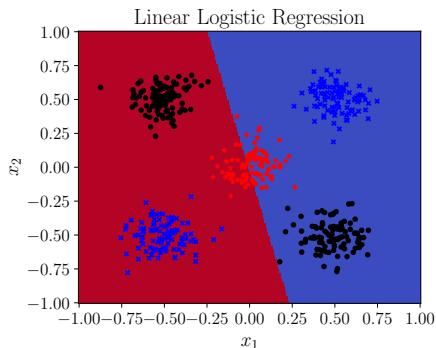
where:

|                                                  |                  |
|--------------------------------------------------|------------------|
| $\mathcal{S}(\cdot)$                             | Softmax function |
| $\mathbf{W} \in \mathbb{R}^{C \times D}$         | Weight matrix    |
| $\mathbf{b} \in \mathbb{R}^C$                    | Bias vector      |
| $\boldsymbol{\theta}(\mathbf{W}, \mathbf{b})$    | Model parameters |
| $\mathbf{a} = \mathbf{W}\mathbf{x} + \mathbf{b}$ | logits vector    |

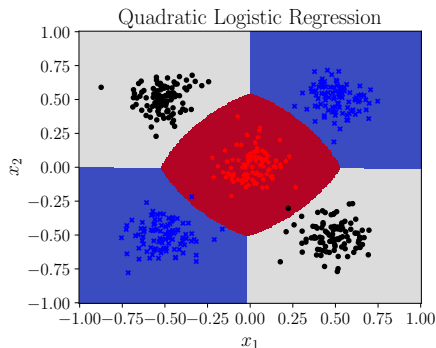
## Augmented Formulation

$$\mathbf{a} = \mathbf{W}\mathbf{x} + \mathbf{b} = [\mathbf{b}, \mathbf{W}] \times [1; \mathbf{x}] \begin{cases} [\mathbf{b}, \mathbf{W}] : \text{Augmented weight vector} \\ [1; \mathbf{x}] : \text{Augmented input feature} \end{cases}$$

# Feature Transformation



(a) Original features



(b) Transformed features

**Figure:** Using feature transformation  $\phi(\mathbf{x}) = [1; x_1; x_2; x_1^2; x_2^2; x_1x_2]$  for reaching nonlinear decision boundary

## NLL

$$\begin{aligned}\text{NLL}(\mathbf{W}) &= -\frac{1}{N} \log p(\mathcal{D}|\mathbf{W}) = -\frac{1}{N} \log \prod_{n=1}^N \prod_{c=1}^C \mu_{nc}^{y_{nc}} = -\frac{1}{N} \sum_{n=1}^N \sum_{c=1}^C y_{nc} \log \mu_{nc} \\ &= \frac{1}{N} \sum_{n=1}^N \mathbb{H}(\mathbf{y}_n, \boldsymbol{\mu}_n)\end{aligned}$$

where:

$$\mu_{nc} = p(y_{nc} = 1 | \mathbf{x}_n, \boldsymbol{\theta}) = \mathcal{S}(f(\mathbf{x}_n; \boldsymbol{\theta}))_c$$

and  $\mathbf{y}_n$  is one-hot encoding of the label.



## Gradient vector

Assume arbitrary input sample  $\mathbf{x}_n$  and row  $\mathbf{w}_j$  in  $\mathbf{W}$  matrix, then:

$$\begin{cases} \nabla_{\mathbf{w}_j} \text{NLL}_n = - \sum_c \frac{\partial}{\partial \mu_{nc}} [y_{nc} \log \mu_{nc}] = - \sum_c \frac{y_{nc}}{\mu_{nc}} \frac{\partial \mu_{nc}}{\partial a_{nj}} \frac{\partial a_{nj}}{\partial \mathbf{w}_j} \\ \frac{\partial \mu_{nc}}{\partial a_{nj}} = \mu_{nc} (\delta_{jc} - \mu_{nj}) \\ \frac{\partial a_{nj}}{\partial \mathbf{w}_j} = \mathbf{x}_j \end{cases}$$

$$\Rightarrow \nabla_{\mathbf{w}_j} \text{NLL}_n = (\mu_{nj} - y_{nj}) \mathbf{x}_n$$

$$\Rightarrow \mathbf{g}(\mathbf{W}) = \frac{1}{N} \sum_{n=1}^N \mathbf{x}_n (\boldsymbol{\mu}_n - \mathbf{y}_n)^T$$

## Hessian Matrix

$$\mathbf{H}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N (\text{diag}(\boldsymbol{\mu}_n) - \boldsymbol{\mu}_n \boldsymbol{\mu}_n^T) \otimes (\mathbf{x}_n \mathbf{x}_n^T)$$

where

$$\left\{ \begin{array}{l} \mathbf{A} = \begin{bmatrix} a_{11} & \dots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \vdots & a_{m,n} \end{bmatrix} \in \mathbb{R}^{m \times n} \\ \mathbf{B} \in \mathbb{R}^{p \times q} \end{array} \right. \Rightarrow \mathbf{A} \otimes \mathbf{B} = \begin{bmatrix} a_{11} \mathbf{B} & \dots & a_{1n} \mathbf{B} \\ \vdots & \ddots & \vdots \\ a_{m1} \mathbf{B} & \dots & a_{mn} \mathbf{B} \end{bmatrix} \in \mathbb{R}^{pm \times qn}$$

and Hessian matrix is PD. Thus for 3 features and 2 classes problem, we have:

$$\mathbf{H}(\mathbf{w}) = \frac{1}{N} \sum_n \begin{bmatrix} \mu_{n1} - \mu_{n1}^2 & -\mu_{n1}\mu_{n2} \\ -\mu_{n1}\mu_{n2} & \mu_{n2} - \mu_{n2}^2 \end{bmatrix} \otimes \begin{bmatrix} x_{n1}x_{n1} & x_{n1}x_{n2} & x_{n1}x_{n3} \\ x_{n2}x_{n1} & x_{n2}x_{n2} & x_{n2}x_{n3} \\ x_{n3}x_{n1} & x_{n3}x_{n2} & x_{n3}x_{n3} \end{bmatrix}$$

## Objective Function

Using MAP estimation we have the following objective function:

$$\text{PNLL}(\mathbf{w}) = \sum_{n=1}^N \mathbb{H}(\mathbf{y}_n, \boldsymbol{\mu}_n) + \lambda \sum_{c=1}^C \|\mathbf{w}_c\|_2^2$$

## Zero Sum Property

At the stationary point for the above regularized formulation, we have:

$$-\sum_{n=1}^N \mathbf{x}_n(\boldsymbol{\mu}_n - \mathbf{y}_n)^T + 2\lambda \mathbf{W}^T = \mathbf{0}$$

$$\text{For column } j \Rightarrow 2\lambda \sum_c w_{cj} = \sum_n \sum_c (y_{nc} - \mu_{nc}) x_{nj} = \sum_n (1 - 1) x_{nj} = 0$$

$$\Rightarrow \sum_c w_{cj} = 0$$

## Un-identifiability

Assume we have a trained MLR model where the posterior probabilities can be computed as:

$$p(y = c | \mathbf{x}, \mathbf{W}) = \frac{\exp(\mathbf{w}_c^T \mathbf{x})}{\sum_{k=1}^C \exp(\mathbf{w}_k^T \mathbf{x})}$$

If we add a constant vector  $\mathbf{v}$  to all rows of  $\mathbf{W}$ , then we have:

$$\begin{aligned} p(y = c | \mathbf{x}, \mathbf{W} + \mathbf{1}\mathbf{v}^T) &= \frac{\exp((\mathbf{w}_c + \mathbf{v})^T \mathbf{x})}{\sum_{k=1}^C \exp((\mathbf{w}_k + \mathbf{v})^T \mathbf{x})} = \frac{\exp(\mathbf{v}) \exp(\mathbf{w}_c^T \mathbf{x})}{\exp(\mathbf{v}) \sum_{k=1}^C \exp(\mathbf{w}_k^T \mathbf{x})} \\ &= p(y = c | \mathbf{x}, \mathbf{W}) \end{aligned}$$

Thus  $\mathbf{W} + \mathbf{1}\mathbf{v}^T$  is also the maximum likelihood estimation and problem is known as un-identifiability.

## Identifiability and MAP

The MAP estimation can solve un-identifiability because:

- Assume weight matrix  $\mathbf{W}$ , then due to zero sum property we have:

$$\sum_{c=1}^C w_{cj} = 0, j = 1, 2, \dots, D + 1$$

- Assume weight matrix  $\mathbf{Z} = \mathbf{W} + \mathbf{1}\mathbf{v}^T$ , then due to zero sum property we have:

$$\sum_{c=1}^C z_{cj} = 0, j = 1, 2, \dots, D + 1$$

Thus:

$$\sum_{c=1}^C z_{cj} = Cv_j + \sum_{c=1}^C w_{cj} = 0 \Rightarrow v_j = 0, j = 1, \dots, D + 1 \Rightarrow \mathbf{v} = \mathbf{0}$$

## Section 4

# Bayesian Logistic Regression

## Laplace Approximation to Posterior

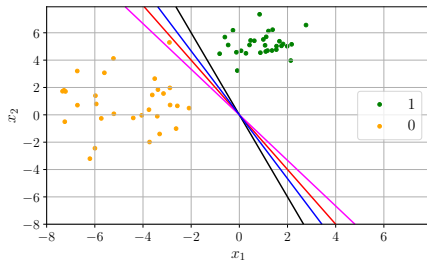
Assume we have a BLR model. Using laplace approximation to posterior, we have:

$$p(\mathbf{w}|\mathcal{D}) \sim \mathcal{N}(\mathbf{w}|\hat{\mathbf{w}}, \mathbf{H}^{-1})$$

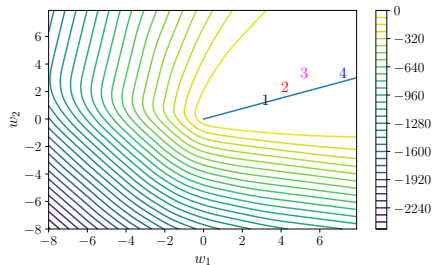
where:

- For MLE we have: 
$$\begin{cases} \hat{\mathbf{w}} = \operatorname{argmin}_{\mathbf{w}} \text{NLL}(\mathbf{w}) \\ \mathbf{H} = \frac{1}{N} \mathbf{X}^T \mathbf{S}(\hat{\mathbf{w}}) \mathbf{X} \end{cases}$$
- For MAP we have: 
$$\begin{cases} \hat{\mathbf{w}} = \operatorname{argmin}_{\mathbf{w}} [\text{NLL}(\mathbf{w}) + \lambda \|\mathbf{w}\|_2^2] \\ \mathbf{H} = \frac{1}{N} \mathbf{X}^T \mathbf{S}(\hat{\mathbf{w}}) \mathbf{X} + 2\lambda \mathbf{I} \end{cases}$$

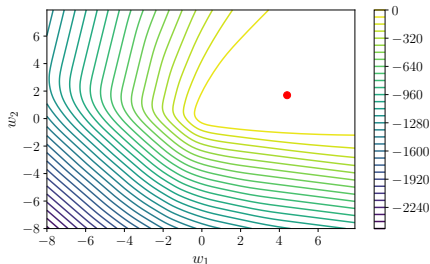
# Bayesian BLR



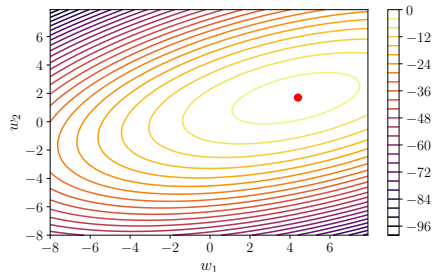
(a) Dataset with four model



(b) Log-likelihood contour plot



(c) Posterior contour plot ( $\mathcal{N}(\mathbf{w}|\mathbf{0}, 100\mathbf{I})$ )



(d) Laplace approximation contour plot



# Approximating the Posterior Predictive

## Posterior Predictive Distribution (PPD)

Posterior predictive distribution is defined as:

$$p(y|\mathbf{x}, \mathcal{D}) = \int p(y|\mathbf{x}, \mathbf{w})p(\mathbf{w}|\mathcal{D})d\mathbf{w}$$

## Point approximate to PPD

In this approach, we ignore the uncertainty in parameters by assuming:

$$p(\mathbf{w}|\mathcal{D}) = \delta(\mathbf{w} - \hat{\mathbf{w}}), \begin{cases} \hat{\mathbf{w}} = \hat{\mathbf{w}}_{mle} \\ \hat{\mathbf{w}} = \hat{\mathbf{w}}_{map} \end{cases}$$

And then approximate PPD as:

$$p(y|\mathbf{x}, \mathcal{D}) \simeq \int p(y|\mathbf{x}, \mathbf{w})\delta(\mathbf{w} - \hat{\mathbf{w}})d\mathbf{w} = p(y|\mathbf{x}, \hat{\mathbf{w}})$$

*Challenge:* Ignoring uncertainty

# Approximating the Posterior Predictive

## Monte Carlo approximate to PPD

In this approach, we draw  $S$  sample from the posterior as  $\mathbf{w}_s \sim p(\mathbf{w}|\mathcal{D})$ , and then approximate it as:

$$p(\mathbf{w}|\mathcal{D}) \simeq \frac{1}{S} \sum_{s=1}^S \delta(\mathbf{w} - \mathbf{w}_s)$$

Then PPD can be approximated as:

$$p(y|\mathbf{x}, \mathcal{D}) \simeq \frac{1}{S} \sum_{s=1}^S \int p(y|\mathbf{x}, \mathbf{w}) \delta(\mathbf{w} - \mathbf{w}_s) d\mathbf{w} = \frac{1}{S} \sum_{s=1}^S p(y|\mathbf{x}, \mathbf{w}_s)$$

*Challenge:* Sampling the posterior at the test time

# Approximating the Posterior Predictive

## Probit Approximation

Assume  $\Phi(a)$  to be normal Gaussian CDF. Then this method uses two following relations:

- $\sigma(a) \simeq \Phi(\lambda a), \lambda^2 = \frac{\pi}{8}$
- $\int \Phi(\lambda a) \mathcal{N}(a|m, \nu) da = \Phi\left(\frac{\lambda m}{(1+\lambda^2\nu)^{\frac{1}{2}}}\right) \simeq \sigma(\kappa(\nu)m), \kappa(\nu) \triangleq (1 + \pi\nu/8)^{-\frac{1}{2}}$

Thus if we define  $a = \mathbf{x}^T \mathbf{w}$ , then we can rewrite PPD as:

$$p(y|\mathbf{x}, \mathbf{w}) = \int p(y|a)p(a|\mathcal{D})da$$

then:

$$\begin{aligned} p(y=1|\mathbf{x}, \mathcal{D}) &\simeq \sigma(\kappa(\nu)m) \\ m &= \mathbb{E}[a] = \mathbf{x}^T \boldsymbol{\mu} \\ \nu &= \mathbb{V}[a] = \mathbb{V}[\mathbf{x}^T \mathbf{w}] = \mathbf{x}^T \boldsymbol{\Sigma} \mathbf{x} \end{aligned}$$

## Intuition

According to probit approximation, the PPD is:

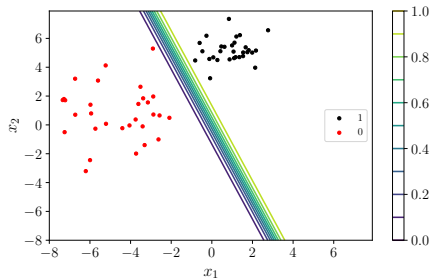
$$p(y = 1|\mathbf{x}, \mathcal{D}) \simeq \sigma(\kappa(\nu)m)$$

We can conclude two important points:

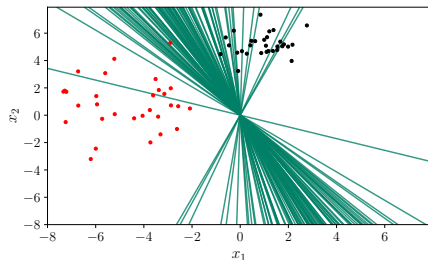
- Because  $0 < \kappa(\nu) < 1$ , then  $\sigma(\kappa(\nu)m)$  is close to 0.5.
- Bayesian setting does not change the decision boundary because:

$$p(y = 1|\mathbf{x}, \mathcal{D}) = 0.5 \Rightarrow \kappa(\nu)m = 0 \Rightarrow m = 0 \Rightarrow \mathbb{E}[\mathbf{w}]^T \mathbf{x} = 0$$

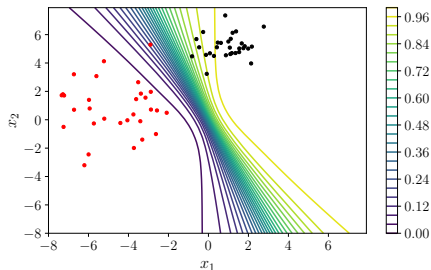
# Bayesian BLR



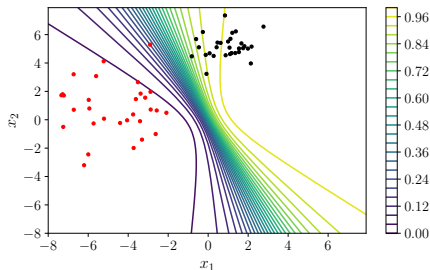
(a) Plug-in approximation  $p(y=1|\mathbf{x}, \hat{\mathbf{w}})$



(b) Samples drawn from  $p(\mathbf{w}|\mathcal{D})$



(c) MC approximation to  $p(y=1|\mathbf{x})$



(d) Probit approximation to  $p(y=1|\mathbf{x})$